ĐẠI HỌC ĐÀ NẴNG
TRƯỜNG ĐẠI HỌC KINH TẾ

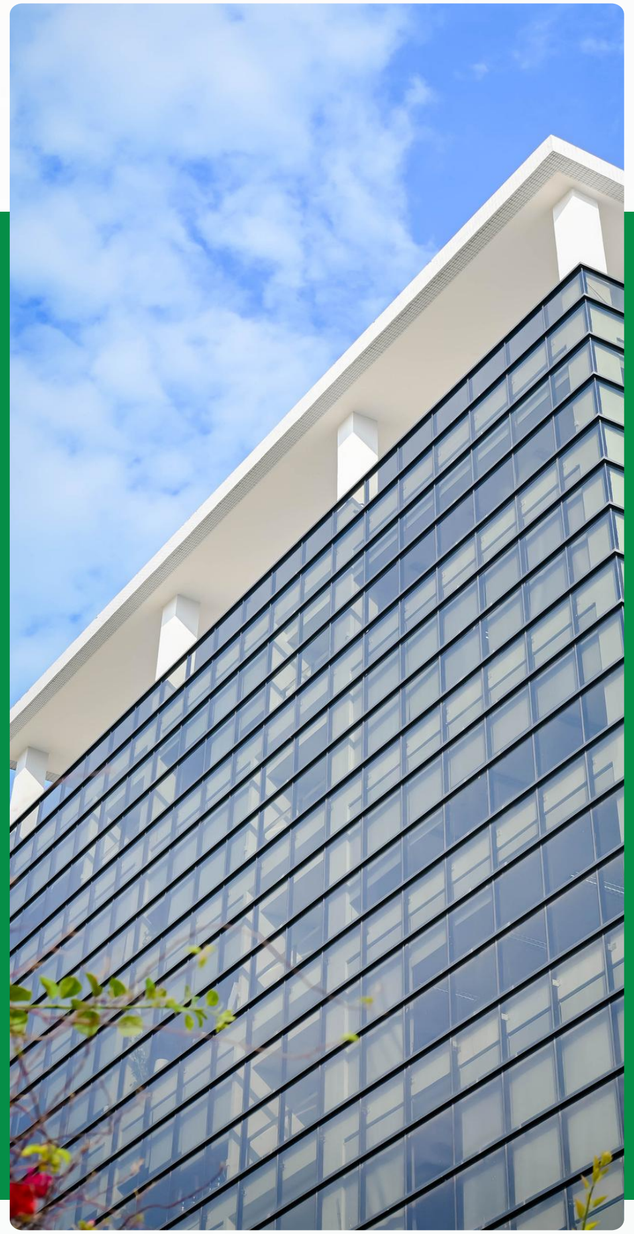
CHƯƠNG 2

KẾT NỐI VỚI CƠ SỞ DỮ LIỆU



LẬP TRÌNH WEB

Khoa Thống kê – Tin học



MỤC TIÊU CHƯƠNG

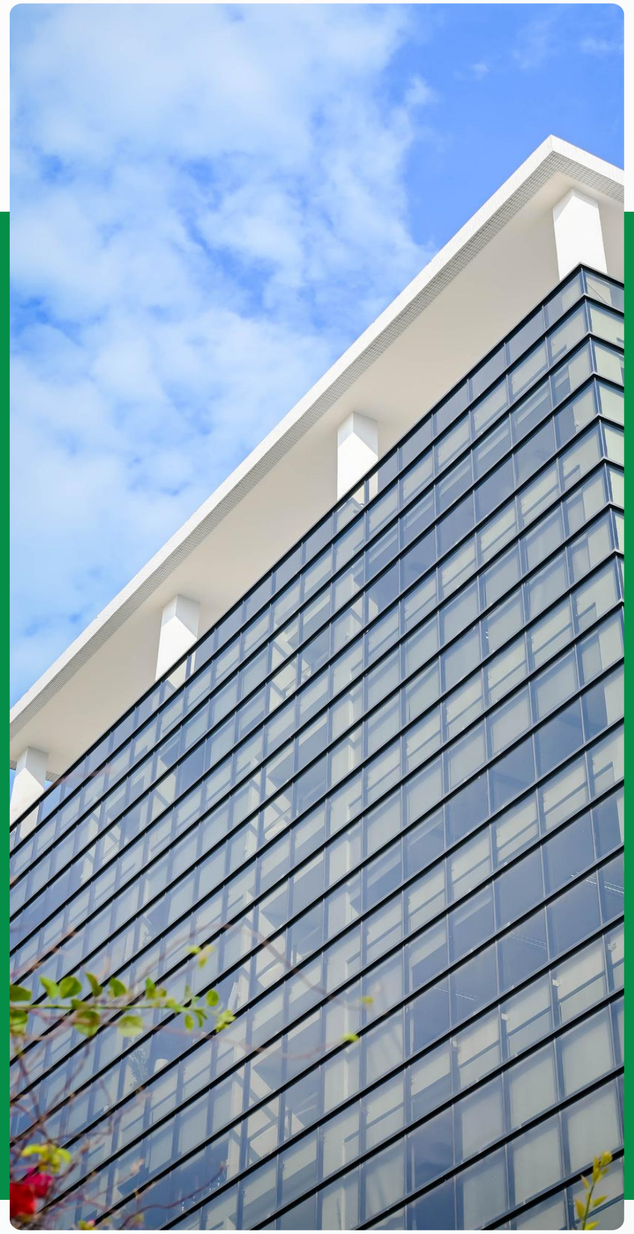
LO1 - Giải thích được vai trò và các loại cơ sở dữ liệu cho ứng dụng web

LO2 - Giải thích được cách thức hoạt động của Django ORM

LO3 - Giải thích được cấu trúc và các thành phần của Model

LO4 - Phân biệt được các loại View

LO5 - Giải thích được cách thức tổ chức của Template



HỌC LIỆU

TL2 - Shaw, B., Badhwar, S., Guest, C., & KS, B. C. (2023), *Web Development with Django: A definitive guide to building modern Python web applications using Django 4*, Packt Publishing Ltd.

2.1 Cơ sở dữ liệu cho ứng dụng Web



Vai trò của cơ sở dữ liệu (CSDL)

- Dữ liệu là phần cốt lõi của hầu hết ứng dụng web
- Các ứng dụng cần lưu trữ dữ liệu ở nơi an toàn, dễ truy cập và luôn sẵn có
  → Cần có CSDL
- CSDL:
  - Tập dữ liệu có cấu trúc nhằm giúp cho việc quản lý thông tin được dễ dàng
  - Có 2 loại: CSDL quan hệ (relational database) và CSDL phi quan hệ (non-relational database)
- Phần mềm để lưu trữ, duy trì và thực hiện các tác vụ trên dữ liệu là Hệ quản trị CSDL

CSDL quan hệ & phi quan hệ

CSDL quan hệ

- Còn gọi là CSDL SQL – Sử dụng ngôn ngữ SQL để quản lý và cập nhật CSDL
- Lưu trữ dữ liệu theo cấu trúc được xác định trước gọi là bảng (quan hệ)

→CSDL gồm nhiều bảng, mỗi bảng có cấu trúc cố định (thuộc tính, kiểu dữ liệu, mối quan hệ với các bảng khác)

→Cấu trúc CSDL này được gọi là lược đồ CSDL (database schema)

CSDL phi quan hệ

- Còn gọi là CSDL NoSQL (not only SQL)
- Lưu trữ dữ liệu phi cấu trúc
- Phù hợp với lượng dữ liệu lớn không theo một cấu trúc xác định
- Thường lưu trữ ở dạng key – value (khóa – giá trị)

→ Chương này chỉ làm việc với CSDL quan hệ

Các thao tác trên CSDL dùng SQL

- Các kiểu dữ liệu cơ bản (trong SQLite3): INTERGER, TEXT, REAL
- Các câu truy vấn SQL để thao tác với dữ liệu:
  - Thêm mới dòng dữ liệu trong bảng: INSERT INTO
  - Đọc dữ liệu từ bảng: SELECT
  - Chỉnh sửa dữ liệu trong bảng: UPDATE
  - Xóa dữ liệu trong bảng: DELETE

→ Có thể sử dụng các truy vấn SQL để làm việc trực tiếp với CSDL quan hệ từ ứng dụng web sử dụng các thư viện liên kết với dữ liệu

2.2 Django ORM



LO2

Giới thiệu về Django ORM

- Việc thao tác trên CSDL dùng SQL có thể dẫn đến các câu lệnh phức tạp khi phải làm việc với nhiều bảng
- Django cung cấp một cấp trừu tượng để thao tác với CSDL dễ dàng hơn → ORM
- Django ORM cho phép chuyển đổi các đoạn mã hướng đối tượng Python thành thao tác trên CSDL thực tế

  → Không cần thiết phải sử dụng câu lệnh SQL
  - Phát triển ứng dụng nhanh hơn
  - Dễ dàng kiểm soát mã nguồn ứng dụng
  - Không cần hoặc ít phải thay đổi mã nguồn khi chuyển sang các CSDL khác nhau

  → Việc chuyển các định nghĩa trên Python thành cấu trúc CSDL thực tế (gọi là **migration**)

Cấu hình kết nối CSDL

- Thiết lập tại tham số DATABASES trong tập tin settings.py của dự án

  Ví dụ:

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'db.sqlite3',
    }
}
```

- Có thể có nhiều kết nối đến các CSDL khác nhau: '**default**' là CSDL mặc định
- Hỗ trợ nhiều loại CSDL khác nhau: từ khóa '**ENGINE**' chỉ ra tên loại CSDL
- Từ khóa **NAME**: tên của CSDL

  *Riêng trong SQLite3 lưu trữ CSDL trong 1 tập tin nên **NAME** có thể chứa đường dẫn đầy đủ đến thư mục lưu tập tin CSDL*

Cấu hình kết nối CSDL

- Các từ khóa khác:
  - USER: Tên tài khoản người dùng (CSDL)
  - PASSWORD: Password đăng nhập của người dùng
  - HOST: Địa chỉ máy chủ CSDL
  - PORT: Số hiệu cổng của máy chủ CSDL
- Một số CSDL phổ biến được hỗ trợ bởi Django:
  - PostgreSQL (‘ENGINE’: 'django.db.backends.postgresql’)
  - MySQL (‘ENGINE’: 'django.db.backends.mysql’)
  - Oracle (‘ENGINE’: 'django.db.backends.oracle’)

2.3 Làm việc với Model



LO3

Tạo mới Model

Các model được lưu trong tập tin **models.py**

Mỗi model tương ứng với một bảng trong cơ sở dữ liệu và các mối quan hệ với các bảng khác

Ví dụ về một lớp model:

```
from django.db import models
class Publisher(models.Model):
    """A company that publishes books."""
    name = models.CharField(
           max_length=50,
           help_text="The name of the Publisher.")
    website = models.URLField(
           help_text="The Publisher's website.")
    email = models.EmailField(
           help_text="The Publisher's email address.")
```

Tạo mới Model

Lớp **Model** được import từ gói **`django.db.models`**
Các model (như Publisher) là lớp con của lớp **Model**
Mỗi thuộc tính của lớp model tương ứng với một trường trong bảng dữ liệu
Một số kiểu dữ liệu của thuộc tính (định nghĩa trong **`django.db.models)`**:

- **CharField**: Văn bản ngắn
- **TextField**: Văn bản dài
- **EmailField**: Là kiểu CharField nhưng văn bản phải theo khuôn dạng địa chỉ email
- **URLField**: Là kiểu CharField nhưng văn phản phải theo khuôn dạng của URL
- **IntegerField**: Số nguyên
- **DateField**: Ngày tháng (tương ứng kiểu datetime.date)
- **DateTimeField**: Thời gian và ngày tháng (tương ứng kiểu datetime.datetime)
- **FloatField**: Số thực (dấu chấm động)

Tạo mới Model

Các tùy chọn về trường dữ liệu

- **help_text**: Văn bản mô tả về trường, có thể tự động thêm vào form của Django
- **max_length**: dùng cho kiểu CharField, để xác định điều dài tối đa của văn bản

Khóa chính

- Sử dụng để phân biệt các bản ghi trong một bảng dữ liệu
- Nếu khóa chính không được khai báo tường minh trong model thì Django sẽ tự động tạo một trường tên là **id** (kiểu **interger**) làm khóa chính với giá trị tự động tăng (***auto-increment***)

  → Khi tự khai báo khóa chính, có thể dùng kiểu dữ liệu AutoField (kiểu interger tự động tăng)

Quan hệ giữa các bảng

- Giúp đảm bảo tính toàn vẹn của CSDL thông qua thiết lập các tham chiếu chéo giữa các bảng
- Các quy tắc của quan hệ còn có thể đảm bảo tính nhất quán và tránh trùng lặp dữ liệu
- Các loại quan hệ: Một - Một, Một - nhiều, Nhiều – Nhiều
- **Quan hệ Một – Nhiều (hay Nhiều – Một):**
  - Nhiều bản ghi ở một bảng tham chiếu đến cùng một bảng ghi ở bảng khác (hoặc chính bảng đó)
  - Khai báo trong Model: Sử dụng kiểu dữ liệu là **ForeignKey**

    Ví dụ:

    ```
    publisher = models.ForeignKey(Publisher, on_delete=models.CASCADE)
    ```

    Lưu ý: tham số on_delete dùng để đảm bảo tính nhất quán của dữ liệu khi xóa bản ghi mà trường này tham chiếu đến, có thể là CASCADE (xóa tất cả các bản ghi tham chiếu), PROTECT (không cho xóa nếu còn bản ghi tham chiếu đến), SET_NULL (đặt giá trị tham chiếu về NULL), SETDEFAULT (đặt giá trị tham chiếu về mặc định)

Quan hệ giữa các bảng

- **Quan hệ Nhiều – Nhiều**
  - Nhiều bản ghi ở một bảng bảng có mối quan hệ với nhiều bản ghi ở bảng khác (hoặc chính bảng đó)
  - Khai báo trong Model: sử dụng kiểu dữ liệu **ManyToManyField**

    Ví dụ:

```
contributors = models.ManyToManyField(
        'Contributor', through="BookContributor")
```

    Trong đó, tham số through chỉ đến bảng trung gian để lưu thông tin về mối quan hệ
- **Quan hệ Một – Một:**
  - Một bản ghi chỉ tham chiếu đến duy nhất một bản ghi ở bảng khác
  - Khai báo trong Model: sử dụng kiểu dữ liệu **OntToOneField**

    Ví dụ:

```
person = models.OneToOneField( Person,
       on_delete=models.CASCADE)
```

Các phương thức của Model

- Thực hiện các thao tác trên dữ liệu của Model
- Gồm: phương thức đặc biệt và phương thức tự định nghĩa
- Phương thức đặc biệt **`__str__():`** trả về chuỗi văn bản biểu diễn đối tượng Model. Phương thức này có thể được ghi đè tùy theo mục đích sử dụng

Tạo lập cơ sở dữ liệu từ các Model

Thực hiện các bước sau:

1. Sử dụng lệnh **`python manage.py makemigrations reviews`** để sinh mã lệnh tạo các bảng dữ liệu
2. Sử dụng lệnh **`python manage.py migrate reviews`** để thực thi các mã lệnh đã sinh ra ở trên để tạo các bảng dữ liệu trong CSDL thực tế

2.4 Thao tác với dữ liệu trong các bảng



LO3

Tạo bản ghi và truy xuất dữ liệu từ bảng

Tạo và lưu bản ghi vào bảng

- Sử dụng phương thức **save()**
- Sử dụng phương thức **objects.create()**
  Đối với các mối quan hệ: thực hiện tạo hoặc truy xuất đến đối tượng có mối quan hệ với đối tượng hiện tại rồi gán vào thuộc tính quan hệ tương ứng

Truy xuất dữ liệu từ bảng

- Truy xuất một hoặc một nhóm bản ghi cụ thể: Sử dụng phương thức **objects.get()** Truy xuất tất cả các bản ghi trong bảng: Sử dụng phương thức **objects .all()**
- Lọc các bản ghi theo điều kiện: Sử dụng phương thức **objects.filter()**
  - Thêm hậu tố ***__toán tử*** (như *__gt, __lt, __gte, __lte*) để thêm điều kiện so sánh giá trị số
  - Thêm hậu tố ***__contains/__icontains*** để thêm điều kiện so khớp chuỗi
- Loại trừ khỏi kết quả: Sử dụng phương thức **objects.exclude()**
- Sắp xếp danh sách truy xuất ra: Sử dụng phương thức **objects.order_by()**

Chỉnh sửa và xóa các bản ghi

Chỉnh sửa và xóa các bản ghi

- Chỉnh sửa các bản ghi : Sử dụng phương thức **update()**
- Xóa các bản ghi: Sử dụng phương thức **delete()**

Thêm và chỉnh sửa hàng loạt các bản ghi

- Thêm hàng loạt: Sử dụng phương thức **objects.bulk_create()**
- Chỉnh sửa hàng loạt: Sử dụng phương thức **objects.bulk_update()**

Thực hiện truy vấn phức tạp sử dụng đối tượng **Q**

- Kết hợp điều kiện **AND**: dùng toán tử **&**
- Kết hợp điều kiện **OR**: dùng toán tử **|**

2.5 Các loại View



View dựa trên hàm (Function-based view)

- Được định nghĩa theo từng hàm, nhận vào tham số là đối tượng request và xuất ra kết quả dạng HTTP Response

  Ví dụ:

```
from django.http import HttpResponse
def home_page(request):
    message = "<html><h1>Welcome to my Website</h1></html>"
    return HttpResponse(message)
```

- **Ưu điểm:** Dễ định nghĩa và sử dụng
- **Hạn chế:** Không thể dùng lại cho nhiều chức năng khác nhau

View dựa trên lớp (Class-based view)

- Được định nghĩa trong các lớp, có thể kế thừa từ các lớp khác

  Ví dụ: *Lớp HomePage sau kế thừa từ lớp* `TemplateView` *đã được định nghĩa sẵn*

```
from django.views.generic import TemplateView
class HomePage(TemplateView):
    template_name = 'home_page.html'
```

  Kết quả xuất ra sẽ như sau:

```
<html><h1>Welcome to my Website</h1></html>
```

- **Ưu điểm:** Có thể khai thác ưu điểm từ tính kế thừa của các lớp để sử dụng lại nhiều lớp đã được định nghĩa trước
- **Hạn chế**: Phức tạp trong tìm hiểu

Cấu hình URL

- URL truy xuất đến View được định nghĩa trong tham số **urlpatterns** của tệp **urls.py**:
  - Sử dụng hàm **path**(): ánh xạ từng đường dẫn riêng lẻ:

```
path('url-path/', views.my_view, name='my-view')
```

Ví dụ: *path('search/', reviews.book_search, name='Book-Search')*

  - Sử dụng hàm **re_path**(): ánh xạ kết hợp với so khớp chuỗi sử dụng biểu thức chính quy

Ví dụ:

```
re_path(r'^url-path/(?P<name>pattern)/$', views.my_view, name='my-view')
```

Khi muốn truyền thêm tham số, như id:

```
re_path(r'^url-path/<int:id>/', views.my_view, name='my-view')
```

Định nghĩa hàm my_views() sẽ như sau:

```
def my_view(request, id):
    return HttpResponse(f"The id is {id}")
```

- Có thể truy xuất lại đường dẫn bằng cách sử dụng **django.urls.reverse()**

2.6 Làm việc với Template



LO5

Khai báo Template

- Thông tin cấu hình sử dụng Template đặt trong biến **TEMPLATES** của tập tin **settings.py**, có dạng:

```
TEMPLATES = [{
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [],
        'APP_DIRS': True,
        'OPTIONS': {
                'context_processors': [
                'django.template.context_processors.debug',
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
        ],},
},]
```

Khai báo Template

Ý nghĩa các từ khóa:

- **BACKEND**: thư viện API để làm việc với Template: Django đã định nghĩa sẵn 2 thư viện là DjangoTemplates và Jinja2. Có thể dùng thư viện bên ngoài
- **DIRS**: Danh sách các thư mục chứa Template theo thứ tự ưu tiên. Có thể tùy biến theo nhu cầu.

  Ví dụ:

  ```
  'DIRS': [os.path.join(BASE_DIR, 'my_templates')]
  ```

- **APP_DIRS**: Chỉ ra có sử dụng Template định nghĩa trong từng App không
- **OPTIONS**: Các thiết đặt cho thư viện API. Danh sách mặc định chứa các bộ xử lý nghĩa cảnh để giúp mã nguồn Python tương tác với các Template để sinh các template HTML động

Ngôn ngữ Template

- **Biến**: được đặt trong cặp đôi dấu ngoặc nhọn **"{{ biến }}"**
  - Khi hiển thị, giá trị của biến sẽ được hiển thị tại vị trí đặt biến
- **Tag**: Các tag dùng để chứa các mã lệnh điều khiển như **if** hay **for**. Các tag này có dạng **{% mã lệnh %}**

  Ví dụ:

```
{% for element in element_list %}
    ...
{% endfor %}
```

- **Ghi chú:** Đặt trong cặp tag `{% comment %} ... {% endcomment %}`
- **Các bộ lọc:** dùng để chỉnh sửa cách biểu biễn của các biến sang các dạng khác. Được ghi sau biến, phân cách bởi dấu "|": `{{ variable|filter }}`

Ví dụ *`{{ variable|lower }}` hay `{{ variable|title}}`: Chuyển các biến kiểu chuỗi thành toàn chữ thường hoặc chữ thường với chữ đầu tiên mỗi từ ghi hoa*

Tính kế thừa trong template

- Tương tự cách kế thừa của các lớp trên Python
- ***Ví dụ:*** *template* ***base.html*** *có dạng như sau*:

```
<!doctype html>
<html lang="en">
<head>
    <meta charset="utf-8">
    <title>Hello World</title>
</head>
    <body>
        <h1>Hello World using Django templates!</h1>
        {% block content %}
        {% endblock %}
    </body>
</html>
```

→ *Template con kế thừa từ* ***base.html*** *sẽ được định nghĩa như sau:*

```
{% extends 'base.html' %}
{% block content %}
<h1>How are you doing?</h1>
{% endblock %}
```

ĐẠI HỌC ĐÀ NẴNG
TRƯỜNG ĐẠI HỌC KINH TẾ

HẾT CHƯƠNG

KHOA THỐNG KÊ – TIN HỌC

